DevEx with Developer Hub

Version 1, 2026-08-04

DevEx with Developer Hub

Introduction

Welcome to this quick course on the ***DevEx with Developer Hub***. This course is the `fourth` in a series of `six` courses about **Application Platform Delivery**:

- *Application Platform Delivery Foundations*
- *Software Factory & Developer Experience*
- *CICD*
- *DevEx with Developer Hub*(*This course*)
- *Application Modernization*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 1.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Container Platform (OCP) Technical Overview or Kubernetes Basics

Lab Environment

Before starting the hands-on exercises, provision the required lab environments from the Red Hat Demo Platform (RHDP).

This learning path uses two OpenShift clusters that represent different stages of a software delivery pipeline. Both environments should be provisioned before you begin the labs.

Provision the Lab Environments

Provision the following RHDP catalog items:

- **Service Enablement: App Platform Software Factory**
- **Service Enablement: App Platform Runtime**

Provisioning each environment typically takes about an hour. Once complete, the cluster URLs and credentials are available from the **My Services** page in RHDP.

[info] | *info.png*

Figure 1. Sample **RHDP My Services** *Screenshot(Click the image to view it in full size in a new browser tab)*

OpenShift Clusters

Environment	Purpose	Used In
Application Platform Software Factory (Primary Cluster)	Development platform where you'll build the internal developer platform, configure CI/CD, and deploy applications.	Modules 2(current), 3, 4, and 5
Application Platform Runtime (Secondary Cluster)	Runtime environment used to demonstrate GitOps-based promotion between environments.	Module 3

The required namespaces have already been created and are ready for use.

Internal Lab Account Credentials

Use the following accounts throughout the learning path.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless instructed otherwise, authenticate to GitLab, Quay, Vault, and other integrated services using **Keycloak Single Sign-On (SSO)**.

Platform Services

The following services are pre-configured and available throughout the labs.

Service	Purpose
Red Hat Build of Keycloak	Provides centralized authentication and Single Sign-On (SSO).
GitLab	Hosts the application source code and Git repositories.
Red Hat Quay	Stores container images built during the labs.
OpenShift GitOps	Continuously synchronizes applications and platform resources from Git.
HashiCorp Vault	Securely stores secrets used by applications and CI/CD pipelines.

All service URLs are available from the **RHDP Lab Information** page.

Developer Hub and the IDP

What's in this section?

In this section, you'll learn how to deploy and configure **Red Hat Developer Hub** as the front door to your Software Factory.

You'll install Developer Hub on OpenShift, create and register your first **Software Template (Golden Path)**, enable the **Software Catalog** for software discovery, and integrate enterprise authentication using **OpenShift OAuth**.

By the end of this section, you'll have a working Internal Developer Portal that allows developers to securely discover software, create new applications through self-service templates, and access the Software Factory through a single, unified experience.

Developer Experience Matters

Why Developer Experience Matters

- Modern software development requires developers to work with numerous tools throughout the Software Development Lifecycle (SDLC).
- A typical developer interacts with:
  - Source code repositories
  - Development environments
  - CI/CD pipelines
  - GitOps deployment tools
  - Container registries
  - Documentation portals
  - Security scanners
  - Monitoring and observability platforms

- Each tool solves a specific problem. However, switching between multiple tools increases the amount of information developers must understand before they can deliver software.
- This additional mental effort is commonly referred to as **developer cognitive load**.
- Platform Engineering aims to reduce this cognitive load by providing standardized workflows and self-service capabilities, allowing developers to spend more time writing applications and less time managing platform complexity.

Great platforms are not measured by the number of tools they provide, but by how simple they make software delivery for developers.

What is Developer Experience?

- *Developer Experience (DevEx)* describes the overall experience developers have while building, testing, deploying, and operating software.
- It encompasses the tools, workflows, platforms, environments, and organizational practices that influence how efficiently developers can deliver applications.

> Developer Experience is the user experience developers have while

> interacting with a platform, its tools, workflows, and supporting services.

- The primary goals of DevEx are to:
  - Reduce friction during software development.
  - Improve developer productivity.
  - Standardize development workflows.
  - Enable developers to focus on delivering business value.

Benefits of Improving Developer Experience

Improving Developer Experience benefits both developers and organizations.

Technical Benefits

- Reduced cognitive load
- Standardized development workflows
- Faster onboarding of new developers
- Faster software delivery
- Improved software quality
- Consistent engineering practices

Business Benefits

- Higher developer productivity
- Faster time to market
- Lower software development costs
- Reduced developer burnout
- Higher employee retention
- Increased innovation

Organizations that invest in Developer Experience often observe improvements in software delivery performance, developer satisfaction, and overall business outcomes.

Developer Experience and Platform Engineering

Developer Experience is closely related to several Platform Engineering concepts.

Platform Engineering

Builds, operates, and maintains the internal platform that enables developers to build and deliver software efficiently.

Internal Developer Platform (IDP)

A collection of standardized tools, automation, services, and workflows that developers use throughout the Software Development Lifecycle.

Developer Experience (DevEx)

Measures how effectively developers can use the Internal Developer Platform to deliver software.

Internal Developer Portal

A centralized, self-service interface where developers discover services, documentation, templates, and workflows required to build and operate applications.

Introducing Red Hat Developer Hub

The Challenge

Modern Software Factories bring together many capabilities, including source control, CI/CD, GitOps, development environments, documentation, observability, and security.

While these capabilities improve software delivery, they also introduce a new challenge for developers.

> If a Software Factory consists of many different tools and platform capabilities, how do developers efficiently discover and use them without learning every underlying technology?

Without a unified experience, developers often need to:

- Navigate multiple web consoles
- Search for project documentation
- Locate source code repositories and build pipelines
- Understand platform-specific implementation details
- Request infrastructure or platform services from different teams

Instead of building software, developers spend valuable time learning tools and navigating platform complexity.

As Software Factories grow, developer productivity depends not only on automation, but also on providing a consistent and discoverable developer

> experience.

The Internal Developer Portal

The solution is an **Internal Developer Portal**.

An Internal Developer Portal provides developers with a single, self-service interface for interacting with the Software Factory.

Rather than accessing separate applications for source control, CI/CD, deployment, documentation, observability, and development environments, developers can access everything from one centralized portal.

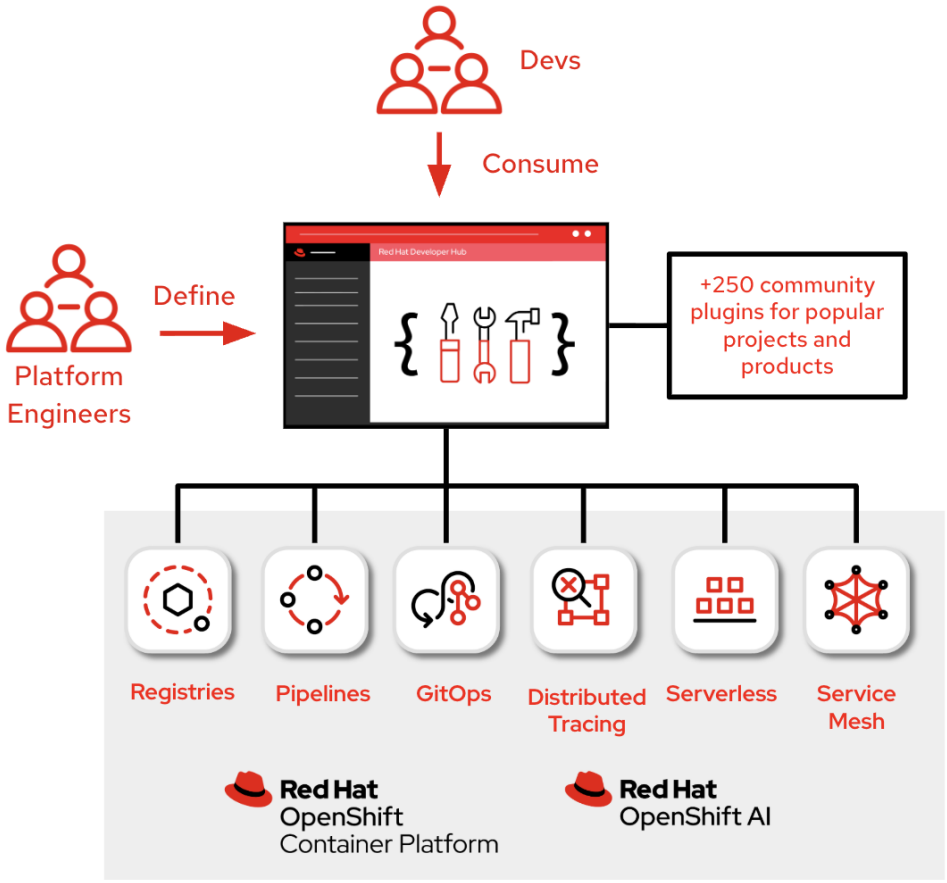

Figure 2. Internal Developer Portal within the Software Factory(Click the image to view it in full size in a new browser tab)

Platform Engineers build and maintain the Software Factory by integrating capabilities such as source control, CI/CD pipelines, GitOps, software catalogs, development environments, observability, and security services.

Developers simply consume these capabilities through the Internal Developer Portal using a consistent self-service experience.

What is Red Hat Developer Hub?

Red Hat Developer Hub (RHDH) is an enterprise-grade Internal Developer Portal built on the CNCF Backstage open source project.

Rather than replacing the tools already used within the Software Factory, Developer Hub integrates them into a single developer experience.

Developers can use Developer Hub to:

- Discover software components, APIs, services, and infrastructure resources
- Create new applications using standardized Software Templates (Golden Paths)
- Launch cloud-based development environments
- Monitor build pipelines and deployment status
- Access project documentation through TechDocs
- Navigate the complete application delivery lifecycle from a single interface

Instead of learning every underlying platform technology, developers interact with a consistent, self-service portal.

Developer Hub within the Software Factory

Developer Hub serves as the entry point into the Software Factory, bringing together platform capabilities into a unified workflow.

Software Factory Capability	**How Developer Hub Integrates**
Source Control	Creates and links application repositories.
Development Environment	Launches cloud-based workspaces using OpenShift Dev Spaces.
Continuous Integration	Displays build pipelines and execution status through Tekton integrations.
Continuous Delivery	Shows deployment status through GitOps integrations such as Argo CD.
Software Catalog	Provides a searchable inventory of software components, APIs, services, and ownership information.
Documentation	Makes project documentation available through TechDocs.
Observability	Integrates monitoring, tracing, and visualization platforms through plugins.

Rather than interacting with each platform independently, developers access these capabilities through one consistent interface.

Business Value

By introducing an Internal Developer Portal, organizations gain:

- A single entry point into the Software Factory
- Improved discoverability of software components and documentation
- Self-service application creation through Software Templates

- Standardized development workflows
- Reduced developer cognitive load
- Faster onboarding and improved developer productivity

Red Hat Developer Hub is not another development tool.

It provides the unified developer experience that enables teams to discover, consume, and interact with the capabilities of a Software Factory through a single self-service portal.

Reflection

▼ *Think about your own organization (Click to expand)*

- How many different tools do developers use during a typical workday?
- Where do developers go to find documentation or application ownership information?
- Which platform capabilities could benefit from self-service access?
- How would a single developer portal improve onboarding and daily development activities?

```
<div class="key-card">
```

Key Takeaway

Red Hat Developer Hub provides a unified, self-service entry point into the Software Factory, allowing developers to discover platform capabilities, create applications, and manage the software delivery lifecycle without needing to learn every underlying tool.

```
</div>
```

Self-Service with Software Templates

The Challenge

Creating a new application often involves much more than writing code.

Developers frequently need to create repositories, configure CI/CD pipelines, prepare deployment manifests, register applications in software catalogs, and configure development environments before they can begin implementing business functionality.

These repetitive setup activities consume valuable development time and often lead to inconsistent project structures across teams.

When every team creates new projects manually, onboarding becomes slower, engineering practices become inconsistent, and maintaining organizational

standards becomes increasingly difficult.

What are Software Templates?

Software Templates, also known as **Golden Paths**, provide standardized project blueprints that automate the creation of new applications.

Rather than starting from an empty repository, developers begin with an organization-approved template that already includes the required project structure, configurations, and platform integrations.

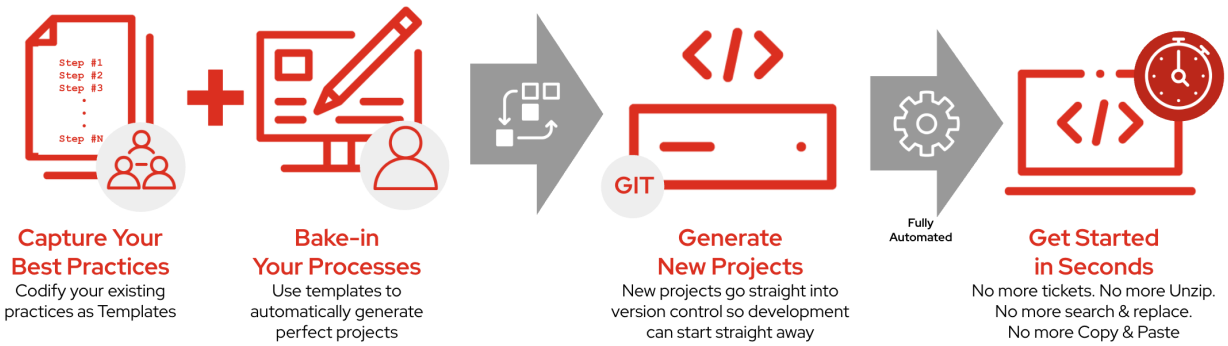

Figure 3. Self-Service Application Creation(Click the image to view it in full size in a new browser tab)

Platform Engineers capture organizational best practices and package them into reusable Software Templates.

Developers simply provide a few project-specific details, while the platform automatically provisions the remaining resources.

How Software Templates Work

Software Templates automate the activities required to bootstrap a new application.

A template can provision:

- Project structure and source code scaffold
- Git repository creation
- CI/CD pipeline configuration
- Deployment manifests, such as Helm charts or Kustomize overlays
- OpenShift Dev Spaces workspace configuration
- Software Catalog registration
- Organizational standards and recommended configurations

Instead of performing these tasks manually, developers complete a simple self-service form, and the platform generates a production-ready project automatically.

Self-Service Project Creation

When a developer selects a Software Template in Red Hat Developer Hub, the platform executes the template and provisions the required resources.

This includes creating repositories, generating source code, configuring pipelines, preparing deployment resources, and registering the application within the Software Factory.

The generated project is immediately available for development, allowing developers to focus on implementing business functionality rather than configuring platform services.

Business Value

By adopting Software Templates, organizations gain:

- Faster application onboarding
- Standardized project structures across development teams
- Self-service application creation
- Consistent security and organizational best practices
- Reduced manual project setup
- Improved developer productivity

Software Templates do not replace developer creativity.

They automate the repetitive foundation of every project, allowing developers to focus on building business functionality while Platform Engineers continuously improve the underlying templates and organizational standards.

Reflection

▼ *Think about your own organization (Click to expand)*

- How is a new application created in your organization today?
- Which project setup activities are still performed manually?
- Which organizational standards could be automated through reusable templates?
- How would self-service project creation improve developer productivity?

```
<div class="key-card">
```

Key Takeaway

Software Templates enable self-service application creation by automating project setup, embedding organizational best practices, and providing developers with a production-ready foundation so they can focus on building business functionality.

Software Template Workflow

Behind the Scenes of a Software Template

Creating a new application from Red Hat Developer Hub appears simple.

A developer selects a **Software Template**, provides a few project details, and clicks **Create**.

Behind that simple interface, however, Developer Hub orchestrates a series of automated tasks that prepare the application for development, testing, deployment, and operations.

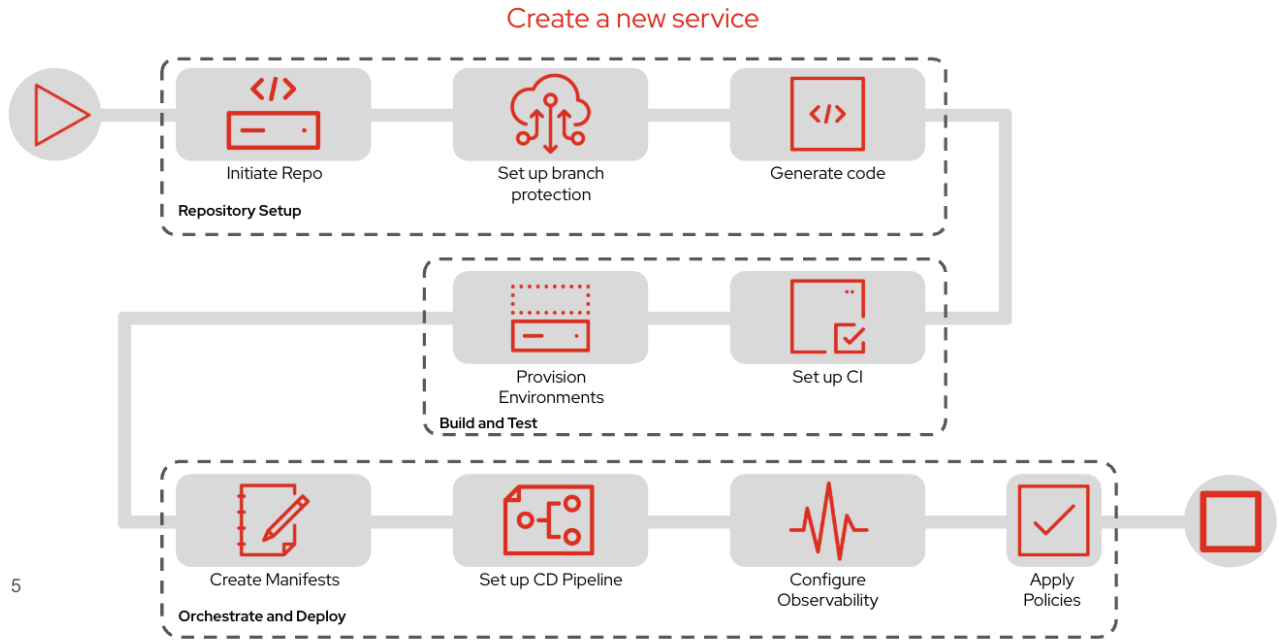

Figure 4. Software Template Workflow (Click the image to view it in full size in a new browser tab)

Rather than generating only application source code, the Software Template establishes the complete foundation required for the application to participate in the organization's Software Factory.

Repository Provisioning

The first stage prepares the application's source repository.

Developer Hub automatically performs tasks such as:

- Creating a new Git repository
- Configuring repository permissions
- Applying branch protection policies
- Generating the initial project structure from the selected template

Instead of manually creating repositories and copying starter projects, developers receive a project

that already follows organizational standards.

Build and Continuous Integration

Next, the template prepares everything required for Continuous Integration (CI).

Typical automation includes:

- Creating build pipelines
- Configuring automated builds
- Preparing testing workflows
- Integrating with the organization's CI platform

Once developers commit their code, the application is immediately ready to be built, tested, and validated through automated pipelines.

Deployment and Operations

The final stage prepares the application for deployment into Kubernetes.

The template automatically generates resources such as:

- Kubernetes deployment manifests
- Helm charts or Kustomize overlays
- GitOps deployment configuration
- Observability integrations
- Security and governance policies

By the time the template finishes, the application is ready to progress through the Software Factory using standardized deployment workflows.

End-to-End Automation

Software Templates automate much more than project creation.

Every generated application already includes the capabilities needed throughout the software delivery lifecycle, including:

- Source control
- Continuous Integration
- Continuous Delivery
- Kubernetes deployment resources
- Observability configuration
- Security and governance

Developers can immediately begin implementing business functionality instead of spending time

assembling platform infrastructure.

Software Templates are maintained by Platform Engineers.

When organizational standards evolve, Platform Engineers update the template once. Every new application generated from that template automatically inherits the latest best practices without requiring developers to manually reconfigure their projects.

Reflection

▼ *Think about your own organization (Click to expand)*

- How much time do developers spend creating repositories and configuring new projects?
- Which setup activities are still performed manually?
- Which of these tasks could be automated using Software Templates?
- How would standardized project generation improve consistency across development teams?

```
<div class="key-card">
```

Key Takeaway

A **Software Template** automates far more than project scaffolding. It provisions repositories, CI/CD pipelines, deployment resources, and platform integrations, allowing developers to focus on building applications while the Software Factory handles the surrounding infrastructure.

```
</div>
```

Software Catalog

The Challenge

As organizations adopt microservices and Platform Engineering, the number of software assets grows rapidly.

Instead of a single application, modern systems often consist of dozens or even hundreds of services, APIs, databases, libraries, and infrastructure resources owned by different teams.

Without a centralized inventory, developers often struggle to answer questions such as:

- Which team owns this service?
- Where is the source code located?
- Which APIs does this application expose?
- Is this application still actively maintained?

- Where can I find its documentation?

Searching across multiple tools wastes valuable development time and increases cognitive load.

When software ownership and documentation are scattered across different systems, developers spend more time searching for information than building new features.

What is the Software Catalog?

The **Software Catalog** provides a centralized inventory of software assets across the organization.

Rather than maintaining spreadsheets or separate documentation portals, Platform Engineers register software assets in the catalog, allowing developers to discover applications, services, APIs, documentation, and ownership information from a single location.

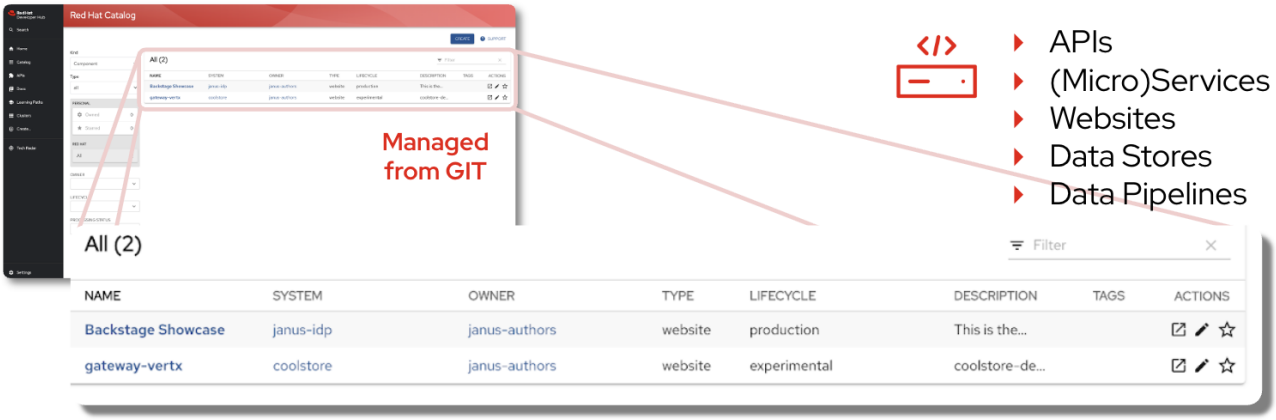

Figure 5. Software Catalog (Click the image to view it in full size in a new browser tab)

The Software Catalog becomes the single source of truth for software components and the relationships between them, making it easier for developers to understand how applications fit within the broader Software Factory.

What Does the Software Catalog Track?

The Software Catalog organizes software into several types of entities.

Entity Type	Description
Components	Individual software components such as microservices, web applications, backend services, frontend applications, or shared libraries.
APIs	REST, GraphQL, gRPC, and other APIs exposed by applications.
Resources	Infrastructure resources such as databases, object storage, message brokers, or Kubernetes resources.
Systems	Collections of related components that together deliver a business application or platform.

Entity Type	Description
Domains	Higher-level business or organizational groupings that contain multiple systems.
Users	Individual developers or engineers associated with software components.
Groups	Development teams responsible for owning and maintaining software.

More Than an Inventory

The Software Catalog provides much more than a list of applications.

Each catalog entry includes valuable metadata that helps developers understand and manage software throughout its lifecycle, including:

- Application owner
- Source code repository
- Lifecycle stage
- System relationships
- API dependencies
- Technical documentation
- CI/CD pipelines
- Deployment status

Instead of searching multiple tools, developers can quickly understand an application's purpose, ownership, and operational status from a single page.

Managed as Code

One of the key principles of Red Hat Developer Hub is that the Software Catalog is **managed as code**.

Each application contains a `catalog-info.yaml` file in its source repository. This file defines metadata such as:

- Component name
- Owner
- Lifecycle
- Type
- Relationships to other software assets

Because this metadata lives alongside the application's source code, updates follow the same Git-based review and approval process used for application development.

This ensures that the catalog remains accurate, version-controlled, and automatically synchronized with the latest state of the application.

Business Value

The Software Catalog benefits both developers and Platform Engineers.

For developers:

- Quickly discover applications, services, and APIs.
- Identify application owners and responsible teams.
- Access repositories, documentation, and deployment information from one location.
- Understand relationships between software components.

For Platform Engineers:

- Maintain an accurate inventory of software assets.
- Improve governance and software ownership.
- Standardize metadata across development teams.
- Enable self-service discovery throughout the organization.

Think of the **Software Catalog** as the organization's software inventory.

Just as a library catalog helps readers quickly find books, the Software Catalog helps developers discover applications, services, APIs, documentation, ownership information, and platform resources from a single interface.

Reflection

▼ *Think about your own organization (Click to expand)*

- How do developers discover existing applications and services today?
- Can you easily identify who owns a particular application or API?
- Where is technical documentation stored?
- How much time is spent searching across different tools for this information?

```
<div class="key-card">
```

Key Takeaway

A **Software Catalog** serves as the central inventory of the Software Factory. It enables developers to discover software assets, understand ownership and relationships, and access documentation, repositories, and operational information through a single self-service portal.

```
</div>
```

Lab 1: Install and configure Red Hat Developer Hub

In this lab, you'll install Red Hat Developer Hub on OpenShift and deploy a minimal portal configuration that will serve as the foundation for the remaining exercises in this module.

By the end of this lab, you will have:

- Installed the Red Hat Developer Hub Operator
- Created a minimal Developer Hub application configuration
- Deployed a Red Hat Developer Hub instance
- Verified that you can access and sign in to the portal

Install the Red Hat Developer Hub Operator

Red Hat Developer Hub is installed and managed using the OpenShift Operator Lifecycle Manager (OLM).

Install the operator from OperatorHub.

1. Open the OpenShift web console.
2. From the navigation menu, select **Ecosystem › Software Catalog**.
3. Search for **Red Hat Developer Hub**.

   

   Select the **Operator Backed** entry. Do **not** select the Helm chart.
4. Click **[Install]**.
5. Accept the following installation settings:
   - **Installation namespace:** `rhdh-operator`
   - **Installation mode:** `All namespaces`
   - **Approval strategy:** `Automatic`
6. Click **[Install]**.
7. Wait until the operator status changes to **Succeeded**.

   [rhdh operator] | *rhdh-operator.png*

   Figure 6. Red Hat Developer Hub Operator (Click the image to view it in full size in a new browser tab)

Create the Developer Hub Project

Deploy Red Hat Developer Hub into a dedicated OpenShift project.

```
oc create namespace setup-rhdh
```

Verify the project was created successfully.

```
oc get namespace setup-rhdh
```

The output should include the `setup-rhdh` namespace.

Create the Application Configuration

Red Hat Developer Hub loads its application settings from an `app-config.yaml` file.

For this training, you'll provide the configuration using a Kubernetes ConfigMap.

Apply the following manifest.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rhdh-config
  namespace: setup-rhdh
data:
  app-config.yaml: |
    app:
      title: Red Hat Developer Hub
    backend:
      reading:
        allow:
          - host: 'gitlab.{openshift_cluster_ingress_domain}'
    auth:
      providers:
        guest:
          dangerouslyAllowOutsideDevelopment: true
```

The `backend.reading.allow` setting allows Red Hat Developer Hub to retrieve Software Catalog information and Software Templates from the GitLab instance used throughout this training.

Guest authentication (`dangerouslyAllowOutsideDevelopment`) is intended only for development and training environments. Production deployments should integrate with an enterprise identity provider such as Red Hat Build of Keycloak.

Verify the ConfigMap

Verify that the ConfigMap was created successfully.

```
oc get configmap rhdh-config -n setup-rhdh
```

The output should include the `rhdh-config` ConfigMap.

Deploy Red Hat Developer Hub

The Red Hat Developer Hub Operator watches for `Backstage` custom resources and automatically provisions the application, database, routes, and supporting Kubernetes resources.

Apply the following manifest.

```
apiVersion: rhdh.redhat.com/v1alpha5
kind: Backstage
metadata:
  name: rhdh
  namespace: setup-rhdh
spec:
  application:
    appConfig:
      mountPath: /opt/app-root/src
      configMaps:
        - name: rhdh-config
    route:
      enabled: true
  database:
    enableLocalDb: true
```

Verify the Deployment

Wait for the Developer Hub pods to become available.

```
oc get pods -n setup-rhdh
```

Verify that both pods are in the `Running` state.

The output should include pods similar to:

- `backstage-rhdh-xxxxxxxx`
- `postgresql-xxxxxxxx`

Next, verify that the Backstage custom resource was created.

```
oc get backstage -n setup-rhdh
```

Finally, retrieve the Developer Hub route.

```
oc get route backstage-rhdh \
  -n setup-rhdh \
  -o jsonpath='https://{.spec.host}{"\n"}'
```

The command returns the URL used to access Red Hat Developer Hub.

Access Red Hat Developer Hub

Open the route URL in your browser.

If you receive an **Application is not available** message, wait a minute for the pods to finish starting and refresh the page.

On the sign-in screen:

1. Select **Guest** and click **[Enter]**.

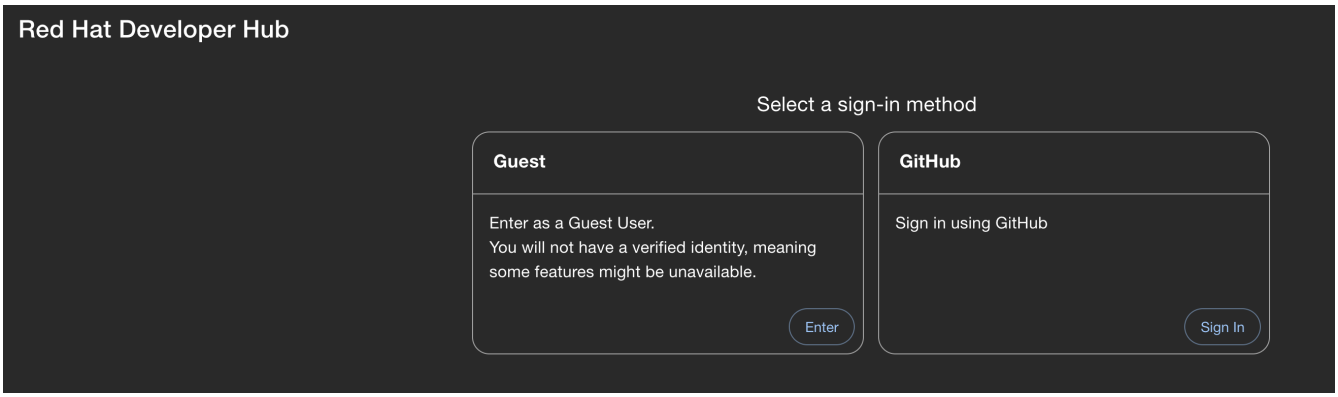

Figure 7. Sample Sign-in Screen (Click the image to view it in full size in a new browser tab)

After signing in, you should see the Red Hat Developer Hub home page.

[home] | *home.png*

Figure 8. Home Page (Click the image to view it in full size in a new browser tab)

Guest authentication is used only for this self-paced training to simplify setup. In production environments, organizations typically authenticate users through enterprise identity providers such as Red Hat Build of Keycloak.

In the next lab, you'll begin customizing Red Hat Developer Hub by importing Software Templates and integrating it with the Software Factory services deployed earlier.

Lab 2: RHDH Templates

In this lab, you'll create and register a **Software Template (Golden Path)** in Red Hat Developer Hub.

Instead of manually creating repositories, pipelines, deployment manifests, and catalog entries, you'll define a reusable template that automates project creation for developers.

By the end of this lab, you will have:

- Created a Git repository for a Software Template
- Defined a Backstage `template.yaml`

- Added a reusable application skeleton
- Registered the template in Red Hat Developer Hub
- Verified the template appears in the **Create** page

Create the Template Repository

Software Templates are stored in Git repositories, just like application source code.

Create a new repository that will contain the Golden Path template.

1. Open your GitLab instance.
2. Navigate to the **Software Factory** group.
3. Create a new **Public** project named `golden-path-quarkus`.
4. Leave **Initialize repository with a README** unchecked.

Clone the repository locally.

```
cd ~/workshop

git clone {gitlab_url}/software-factory/golden-path-quarkus.git

cd golden-path-quarkus
```

Understand the Template Structure

A Software Template contains two major parts:

- **template.yaml** — describes the workflow executed by Developer Hub
- **skeleton/** — contains the files copied into newly generated applications

The repository structure will look similar to:

```
golden-path-quarkus/
├── template.yaml
└── skeleton/
    ├── devfile.yaml
    ├── pom.xml
    ├── src/
    ├── catalog-info.yaml
    ├── tekton/
    └── gitops/
```

Everything inside the `skeleton` directory becomes the starting point for every project created from this template.

Create the Template Definition

Create a file named `template.yaml` in the repository root.

Paste the following template definition.

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: quarkus-golden-path
  title: Quarkus Application (Golden Path)
  description: |
    Create a Quarkus application with CI/CD pipeline, GitOps deployment,
    Dev Spaces workspace, and OpenTelemetry instrumentation pre-configured.
  tags:
    - quarkus
    - java
    - recommended
spec:
  owner: platform-team
  type: service
  parameters:
    - title: Application Details
      required:
        - name
        - owner
      properties:
        name:
          title: Application Name
          type: string
          pattern: "^[a-z0-9-]+$"
        description:
          title: Description
          type: string
        owner:
          title: Owner
          type: string
          ui:field: OwnerPicker

    - title: Repository
      required:
        - repoUrl
      properties:
        repoUrl:
          title: Repository Location
          type: string
          ui:field: RepoUrlPicker
          ui:options:
            allowedHosts:
              - {gitlab_url}
  steps:
```

```
    - id: fetch-skeleton
      name: Fetch Skeleton
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          description: ${{ parameters.description }}
          owner: ${{ parameters.owner }}

    - id: publish
      name: Publish to GitLab
      action: publish:gitlab
      input:
        repoUrl: ${{ parameters.repoUrl }}
        description: ${{ parameters.description }}
        defaultBranch: main

    - id: register
      name: Register in Catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps['publish'].output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml

  output:
    links:
      - title: Repository
        url: ${{ steps['publish'].output.remoteUrl }}
      - title: Open in Catalog
        icon: catalog
        entityRef: ${{ steps['register'].output.entityRef }}
```

This template performs three automated tasks:

- Generates a project from the skeleton
- Publishes the generated project to GitLab
- Registers the new application in the Software Catalog

Create the Catalog Entity

Inside the `skeleton` directory, create a file named `catalog-info.yaml`.

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: ${{ values.name }}
  description: ${{ values.description }}
  annotations:
```

```
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: ${{ values.name }}-dev
    janus-idp.io/tekton: ${{ values.name }}
spec:
  type: service
  lifecycle: production
  owner: ${{ values.owner }}
```

Every application generated from this template will automatically include this catalog metadata.

This allows Red Hat Developer Hub to understand:

- Application ownership
- Lifecycle
- Documentation
- CI/CD integrations
- Deployment relationships

Commit and Push the Template

Commit the new template repository.

```
git add .

git commit -m "feat: initial Golden Path template"

git push origin main
```

Register the Template in Developer Hub

Developer Hub discovers Software Templates through **catalog locations** configured in `app-config.yaml`.

Update the existing `rhdh-config` ConfigMap created in the previous lab.

1. Switch to the **setup-rhdh** project.
2. Open **Workloads** → **ConfigMaps**.
3. Select **rhdh-config**.
4. Choose **Actions** → **Edit ConfigMap**.
5. Add the following `catalog` section under `data.app-config.yaml`.

```
catalog:
  locations:
    - type: url
      target: '{gitlab_url}/software-factory/golden-path-quarkus/-
```

```
/raw/main/template.yaml'
      rules:
        - allow: [Template]
```

Save the ConfigMap.

The Red Hat Developer Hub Operator automatically rolls out the updated configuration and restarts the application.

Verify the Template

Wait until the Developer Hub pod returns to the **Running** state.

Open Red Hat Developer Hub.

Navigate to:

Create

You should now see the **Quarkus Golden Path** template.

[rhdh quarkus golden path template] | *rhdh-quarkus-golden-path-template.png*

Figure 9. Quarkus Golden Path Template

Select the template to verify its metadata and input form are displayed correctly.

Lab 3: RHDH Integration with Dev Spaces

Add a Dev Spaces link to catalog components so developers can open a workspace directly from the component page.

Add the Dev Spaces URL Annotation

In the `catalog-info.yaml` skeleton, add the Dev Spaces annotation:

```
metadata:
  annotations:
    che.eclipse.org/che-server: {devspaces_url}
```

Do not forget to commit and push the changes to the Git repository.

```
git add catalog-info.yaml
git commit -m "feat: add dev spaces annotation"
git push origin main
```

When a component has this annotation, the Developer Hub UI shows a "Dev Spaces" link that opens

a workspace for the component's repository.

Configure the Dev Spaces Plugin

Add the Dev Spaces plugin configuration to **rhdh-config** ConfigMap. Under `data.app-config.yaml`, keep your existing `app`, `auth` and other settings and add the `devSpaces` block below. Do not replace the whole file—merge the new section with what is already there.

```
    devSpaces:
      defaultUrl: {devspaces_url}
```

Update the ConfigMap and wait for the pod to restart.

Lab 4: Configure OpenShift OAuth Authentication

In this lab, you'll replace the temporary Guest authentication used during installation with OpenShift OAuth authentication.

This allows developers to sign in using their OpenShift identities, providing a production-like authentication experience for Red Hat Developer Hub.

By the end of this lab, you will have:

- Created an OpenShift OAuth client
- Configured Developer Hub to authenticate using OpenShift OAuth
- Mounted authentication secrets into the Backstage application
- Verified user sign-in with OpenShift credentials
- Tested the Golden Path template using an authenticated user

Create an OpenShift OAuth Client

Developer Hub authenticates users through an OAuth client registered with OpenShift.

Create the OAuth client.

```
oc create -f - <<EOF
apiVersion: oauth.openshift.io/v1
kind: OAuthClient
metadata:
  name: rhdh-oauth-client
grantMethod: auto
redirectURIs:
  - https://backstage-rhdh-setup-
rhdh.{openshift_cluster_ingress_domain}/api/auth/oidc/handler/frame
secret: GENERATE_A_RANDOM_SECRET
EOF
```

Generate a secure client secret before running the command.

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Replace `GENERATE_A_RANDOM_SECRET` with the generated value.

Create the Authentication Secret

Store the OAuth client information in a Kubernetes Secret.

```
oc create secret generic rhdh-auth-secret \
  -n setup-rhdh \
  --from-literal=AUTH_OIDC_CLIENT_ID=rhdh-oauth-client \
  --from-literal=AUTH_OIDC_CLIENT_SECRET=GENERATE_A_RANDOM_SECRET \
  --from-literal=AUTH_OIDC_METADATA_URL=https://oauth
-openshift.{openshift_cluster_ingress_domain}/.well-known/openid-configuration
```

Use the same client secret that was configured when creating the OAuth client.

Update the Developer Hub Configuration

The initial installation used the Guest authentication provider.

Update the existing `app-config.yaml` to use OpenShift OAuth instead.

1. Open the **setup-rhdh** project.
2. Navigate to **Workloads** → **ConfigMaps**.
3. Select **rhdh-config**.
4. Choose **Actions** → **Edit ConfigMap**.
5. Replace the Guest authentication configuration with the following OIDC configuration.

```
signInPage: oidc

auth:
  session:
    secret: MY_SESSION_SECRET

  environment: production

  providers:
    oidc:
      production:
        clientId: ${AUTH_OIDC_CLIENT_ID}
        clientSecret: ${AUTH_OIDC_CLIENT_SECRET}
        metadataUrl: ${AUTH_OIDC_METADATA_URL}
```

```
        prompt: auto
```

Generate a session secret using:

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Replace `MY_SESSION_SECRET` with the generated value.

Click **Save**.

The `${AUTH_OIDC_*}` values are resolved from the Kubernetes Secret that will be mounted into the Backstage application in the next step.

Mount the Authentication Secret

Mount the authentication Secret so Developer Hub can access the OAuth client credentials.

1. Switch to the **setup-rhdh** project.
2. Navigate to **Operators** → **Installed Operators**.
3. Open **Red Hat Developer Hub**.
4. Select the **Backstage** tab.
5. Open the **rhdh** instance.
6. Click **Actions** → **Edit Backstage**.
7. Under `spec.application`, add:

```
extraEnvs:
  secrets:
    - name: rhdh-auth-secret
```

1. Save the resource.

The Red Hat Developer Hub Operator automatically reconciles the change and restarts the Backstage pod.

Verify OpenShift Sign-In

Wait until the Backstage pod returns to the **Running** state.

1. Open the Developer Hub route.
2. Click **OIDC** on the login page.
3. Sign in using your OpenShift credentials.

After successful authentication, you should be redirected to the Developer Hub home page as your

authenticated OpenShift user.

Guest authentication is no longer available after enabling OpenShift OAuth.

Test the Golden Path Template

Verify that authenticated users can create new applications.

1. Open **Create › Templates**.
2. Select **Quarkus Application (Golden Path)**.
3. Enter the required project information.
4. Click **[Create]**.

Verify that:

- A new GitLab repository is created.
- The application is registered in the Software Catalog.
- The generated repository contains:
  - Devfile
  - Tekton pipeline
  - GitOps manifests
- The catalog component provides a link to launch a Dev Spaces workspace.

Verify the Deployment

Confirm the authentication flow is working correctly by checking:

- You are signed in using your OpenShift identity.
- Guest authentication is no longer available.
- The Golden Path template successfully provisions a new project.
- The generated project appears in the Software Catalog.

```
<div class="key-card">
```

Key Takeaway

Red Hat Developer Hub now uses OpenShift OAuth for authentication, allowing developers to securely access Software Templates and the Software Catalog using their enterprise identity while maintaining a production-ready authentication model.

```
</div>
```